

OS Mid-2 Important Q&A

1) Paging & Demand paging and Page Replacement Algorithms.

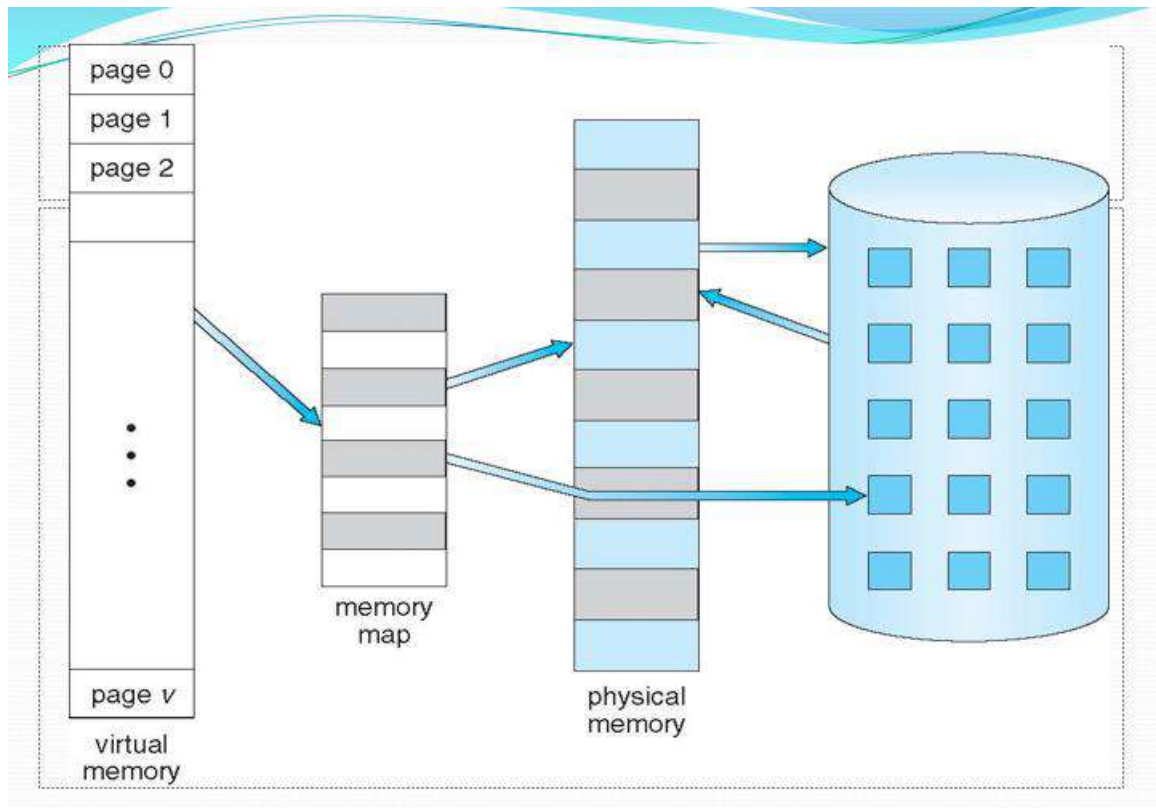
Ans:-

Introduction

- Program to be executed need to be in physical memory- limits the size of the program to size of memory.
- Consider the following cases where entire prog. is not needed-
 - Code that handles error conditions.
 - Size of arrays, lists, tables.
 - Certain options and features of prog. used rarely.
 - Even the case where the total program is needed but not all at the same time.

- Virtual Memory
- Benefits of loading the program partially are :
 - Free of physical memory constraint.
 - More programs can run at the same time.
 - Load or swap will be easier.
- Virtual memory involves separation of logical address from physical address.
- **Virtual Memory** is a storage mechanism which offers user an illusion of having a very big main memory.

In Virtual memory, the user can store processes with a bigger size than the available main memory.



Demand paging

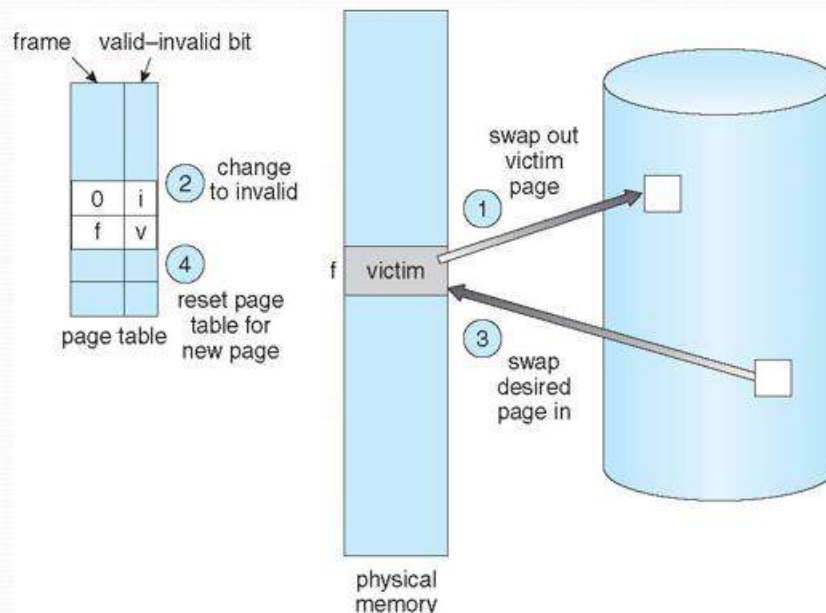
- **Demand paging** : Pages are loaded when they are demanded during program execution, pages never accessed are never loaded.
- Similar to paging, demand paging allocates physical memory to the process when there is a request for execution-swapping.
- But instead of swapping the process, we swap **pages-pager**.

- In demand paging some pages of process will be in physical memory and some on the disk.
- To distinguish b/w these two we use valid/invalid bit in the page table.
- When it is set to valid indicates that page is legal and in physical memory
- Invalid indicates either it is not valid or is valid and is currently on disk.

Basic Page Replacement

1. Find the location of the desired page on disk
2. Find a free frame:
 - If there is a free frame, use it
 - If there is no free frame, use a page replacement algorithm to select a **victim frame**
 - Write victim frame to disk if dirty
3. Bring the desired page into the (newly) free frame; update the page and frame tables
4. Continue the process by restarting the instruction that caused the trap

Page Replacement

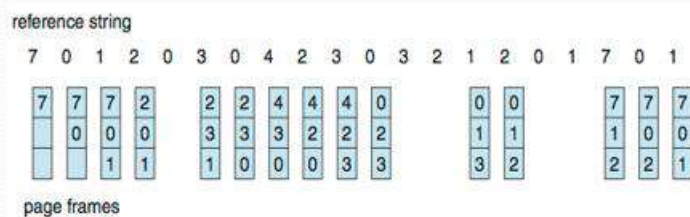


Page and Frame Replacement Algorithms

- **Frame-allocation algorithm** determines
 - How many frames to give each process
 - Which frames to replace
- **Page-replacement algorithm**
 - Want lowest page-fault rate on both first access and re-access
- Evaluate algorithm by running it on a particular string of memory references (reference string) and computing the number of page faults on that string
 - String is just page numbers, not full addresses
 - Repeated access to the same page does not cause a page fault
 - Results depend on number of frames available

First-In-First-Out (FIFO) Algorithm

- Reference string:
7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1
- 3 frames (3 pages can be in memory at a time per process)

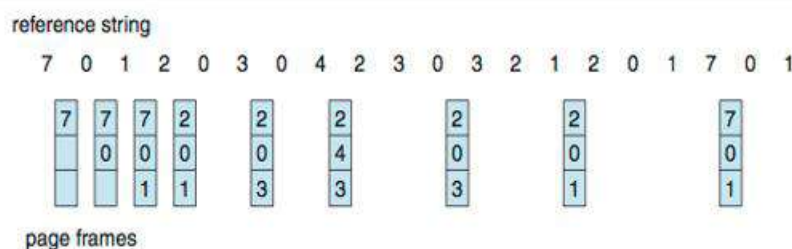


15 page faults

- The page fault rate may increase with increase in number of frames-**Belady's anomaly**.

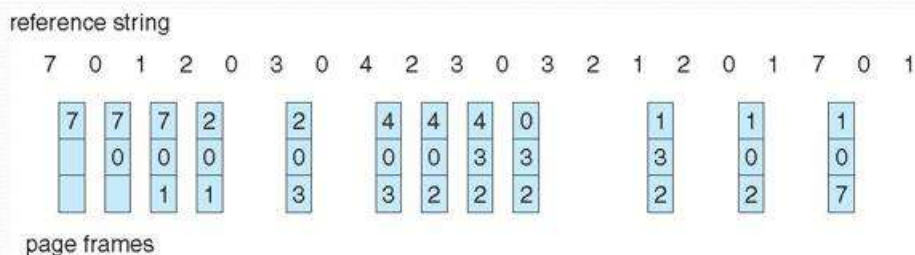
Optimal Algorithm

- Replace page that will not be used for longest period of time
 - 9 is optimal for the example
- How do you know this?
 - Can't read the future
- Used for measuring how well your algorithm performs



Least Recently Used (LRU) Algorithm

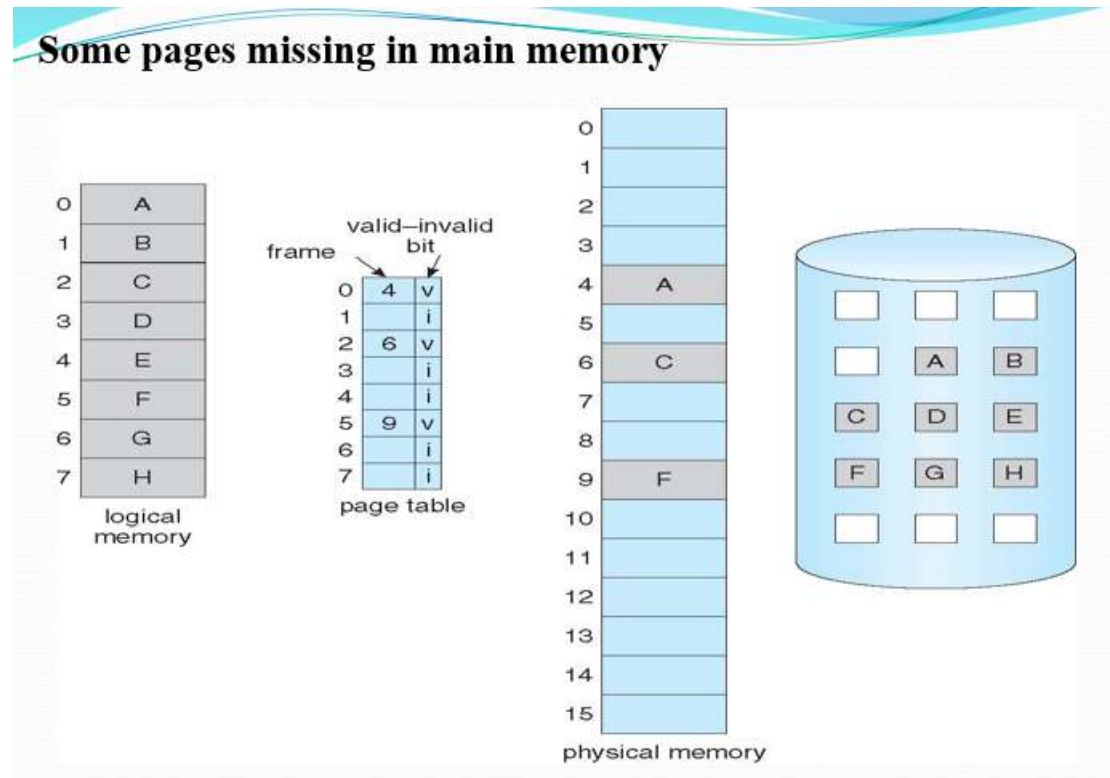
- Use past knowledge rather than future
- Replace page that has not been used in the most amount of time
- Associate time of last use with each page



- 12 faults – better than FIFO but worse than OPT
- Generally good algorithm and frequently used
- But how to implement?

2) What are Page Faults and explain how to handle them?

Ans:-



Page Fault

- If there is a reference to a page, first reference to that page will trap to operating system:

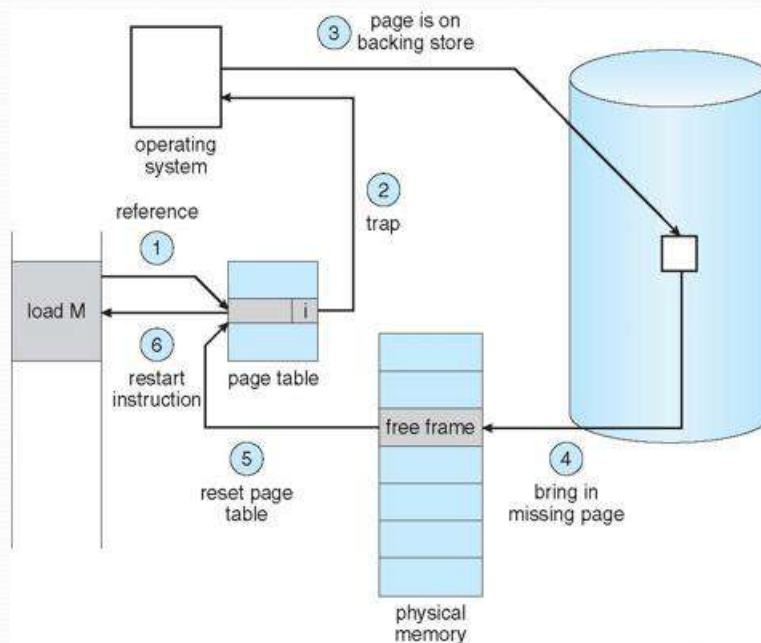
page fault

1. Operating system looks at another table to decide:
 - Invalid reference $\Rightarrow$ abort
 - Just not in memory
2. Find free frame
3. Swap page into frame via scheduled disk operation
4. Reset tables to indicate page now in memory
Set validation bit = **v**
5. Restart the instruction that caused the page fault

• The procedure for handling this page fault is:

1. check the page table to determine whether the reference was a valid or an invalid memory access.
2. If the reference was invalid, we terminate the process. If it was valid but not in main memory, we now load it in.
3. We find a free frame by taking one from the free-frame list.
4. We schedule a disk operation to read the desired page into the frame.
5. When the disk read is complete, we modify the internal reference bit in page table.
6. We restart the instruction that was interrupted by the trap.

Steps in Handling a Page Fault



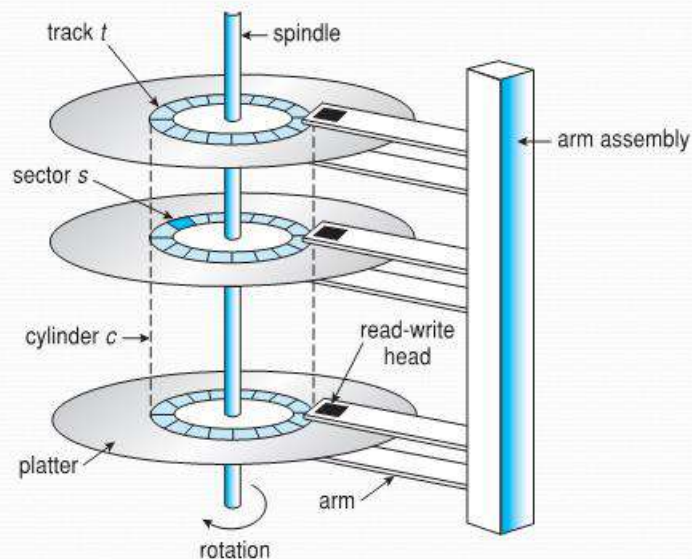
3) Disk scheduling Algorithms and Mass Storage Structure and Swap Space Management.

Ans:-

Overview of Mass Storage Structure

- **Magnetic disks** provide bulk of secondary storage of modern computers
 - Drives rotate at 60 to 250 times per second
 - **Transfer rate** is rate at which data flow between drive and computer
 - **Positioning time (random-access time)** is time to move disk arm to desired cylinder (**seek time**) and time for desired sector to rotate under the disk head (**rotational latency**)
 - **Head crash** results from disk head making contact with the disk surface.

Moving-head Disk Mechanism



Magnetic Tape

- Was early secondary-storage medium
 - Evolved from open spools to cartridges
- Relatively permanent and holds large quantities of data
- Access time slow
- Random access ~1000 times slower than disk
- Mainly used for backup, storage of infrequently-used data, transfer medium between systems
- Kept in spool and wound or rewound past read-write head
- Once data under head, transfer rates comparable to disk.

Disk Structure

- Disk drives are addressed as large 1-dimensional arrays of **logical blocks**, where the logical block is the smallest unit of transfer
 - 512 to 1024 bytes
 - The 1-dimensional array of logical blocks is mapped into the sectors of the disk sequentially
 - Sector 0 is the first sector of the first track on the outermost cylinder
-
- Mapping proceeds in order through that track, then the rest of the tracks in that cylinder, and then through the rest of the cylinders from outermost to innermost
 - Logical to physical address should be easy
 - Except for bad sectors
 - Non-constant # of sectors per track via constant angular velocity
 - In order to keep constant transfer rate:
 - Use constant linear velocity (CLV)-drive increases its rotation speed as the head moves from the outer to the inner tracks to keep the same rate of data moving under the head
 - constant angular velocity (CAV):the disk rotation speed can stay constant; in this case, the density of bits decreases from inner tracks to outer tracks to keep the data rate constant.

Disk Attachment

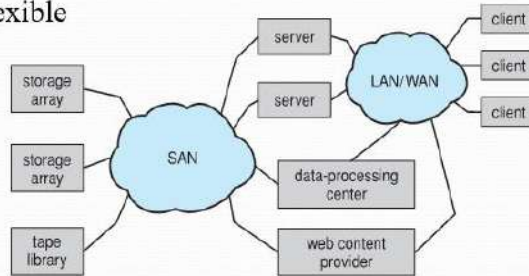
- Computers access disk storage in two ways. One way is via I/O ports (or **host-attached storage**); This is common on small systems. The other way is via a remote host in a distributed file system; this is referred to as **network-attached storage**
- Host-attached storage accessed through I/O ports talking to I/O busses
- SCSI itself is a bus, up to 16 devices on one cable, **SCSI initiator** requests operation and **SCSI targets** perform tasks
 - Each target can have up to 8 **logical units** (disks attached to device controller)
- FC is high-speed serial architecture
 - Can be switched fabric with 24-bit address space – the basis of **storage area networks (SANs)** in which many hosts attach to many storage units
- I/O directed to bus ID, device ID, logical unit (LUN)

Storage Array

- Can just attach disks, or arrays of disks
- Storage Array has controller(s), provides features to attached host(s)
 - Ports to connect hosts to array
 - Memory, controlling software (sometimes NVRAM, etc)
 - A few to thousands of disks
 - RAID, hot spares, hot swap (discussed later)
 - Shared storage -> more efficiency
 - Features found in some file systems
 - Snapshots, clones, thin provisioning, replication, deduplication, etc

Storage Area Network

- Common in large storage environments
- Multiple hosts attached to multiple storage arrays - flexible

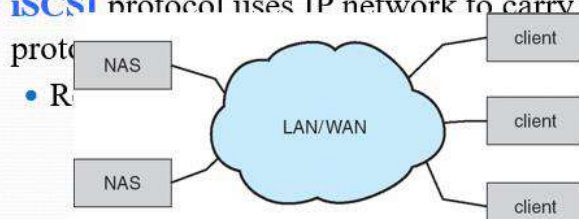


Storage Area Network (Cont.)

- SAN is one or more storage arrays
 - Connected to one or more Fibre Channel switches
- Hosts also attach to the switches
- Storage made available via **LUN Masking** from specific arrays to specific servers
- Easy to add or remove storage, add new host and allocate it storage
 - Over low-latency Fibre Channel fabric
- Why have separate storage networks and communications networks?
 - Consider iSCSI, FCOE

Network-Attached Storage

- Network-attached storage (**NAS**) is storage made available over a network rather than over a local connection (such as a bus)
 - Remotely attaching to file systems
- NFS and CIFS are common protocols
- Implemented via remote procedure calls (RPCs) between host and storage over typically TCP or UDP on IP network
- **iSCSI** protocol uses IP network to carry the SCSI



Disk Scheduling

- The operating system is responsible for using hardware efficiently — for the disk drives, this means having a fast access time and disk bandwidth
- Minimize seek time- The seek time is the time for the disk arm to move the heads to the cylinder containing the desired sector
- The rotational latency is the additional time for the disk to rotate the desired sector to the disk head
- Disk **bandwidth** is the total number of bytes transferred, divided by the total time between the first request for service and the completion of the last transfer

Disk Scheduling (Cont.)

- There are many sources of disk I/O request
 - OS
 - System processes
 - Users processes
- I/O request includes input or output mode, disk address, memory address, number of sectors to transfer
- OS maintains queue of requests, per disk or device
- Idle disk can immediately work on I/O request, busy disk means work must queue
 - Optimization algorithms only make sense when a queue exists

Disk Scheduling

- First Come-First Serve (FCFS)
- Shortest Seek Time First (SSTF)
- Elevator (SCAN)
- Circular SCAN (C-SCAN)
- LOOK
- C-LOOK

Disk Scheduling (Cont.)

- Note that drive controllers have small buffers and can manage a queue of I/O requests (of varying “depth”)
- Several algorithms exist to schedule the servicing of disk I/O requests
- The analysis is true for one or many platters
- We illustrate scheduling algorithms with a request queue (0-199)

98, 183, 37, 122, 14, 124, 65, 67

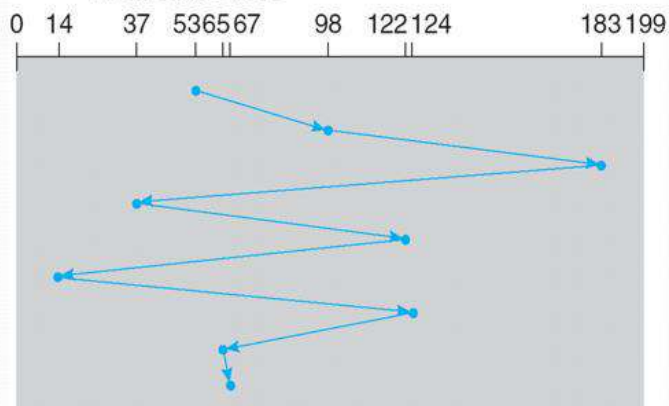
Head pointer 53

FCFS

Illustration shows total head movement of 640 cylinders

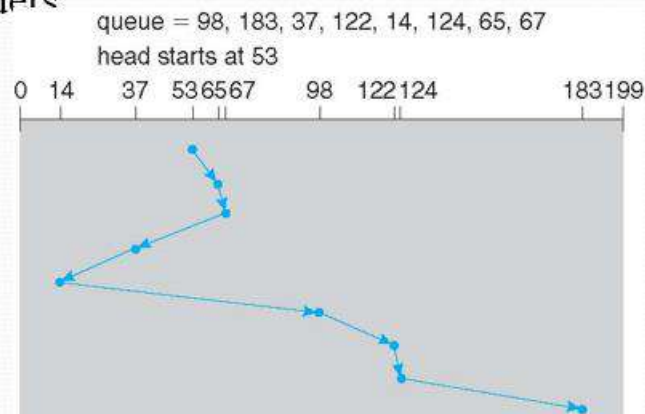
queue = 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53



SSTF

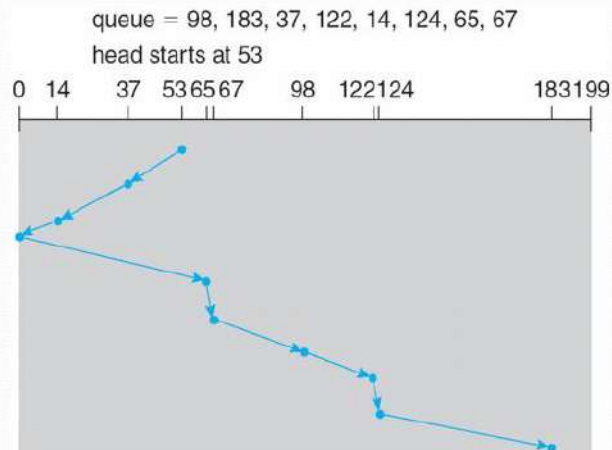
- Shortest Seek Time First selects the request with the minimum seek time from the current head position
- SSTF scheduling is a form of SJF scheduling; may cause starvation of some requests
- Illustration shows total head movement of 236 cylinders



SCAN

- The disk arm starts at one end of the disk, and moves toward the other end, servicing requests until it gets to the other end of the disk, where the head movement is reversed and servicing continues.
- **SCAN algorithm** Sometimes called the **elevator algorithm**
- Illustration shows total head movement of 236 cylinders.

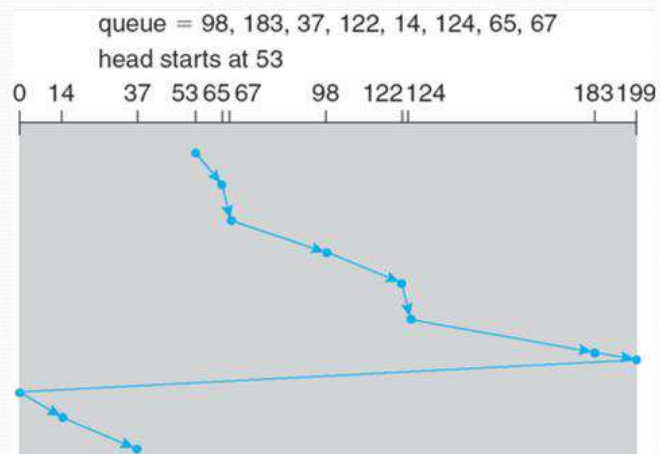
SCAN (Cont.)



C-SCAN

- Provides a more uniform wait time than SCAN
- The head moves from one end of the disk to the other, servicing requests along the way.
 - When the head reaches the other end, however, it immediately returns to the beginning of the disk, without servicing any requests on the return trip
- Treats the cylinders as a circular list that wraps around from the last cylinder to the first one

C-SCAN (Cont.)



LOOK

- It is similar to the SCAN disk scheduling algorithm except for the difference that the disk arm in spite of going to the end of the disk goes only to the last request to be serviced in front of the head and then reverses its direction from there only. Thus it prevents the extra delay which occurred due to unnecessary traversal to the end of the disk.

98,183,37,122,14,124,65,67(Higher Direction)

Read/Write arm is at 50

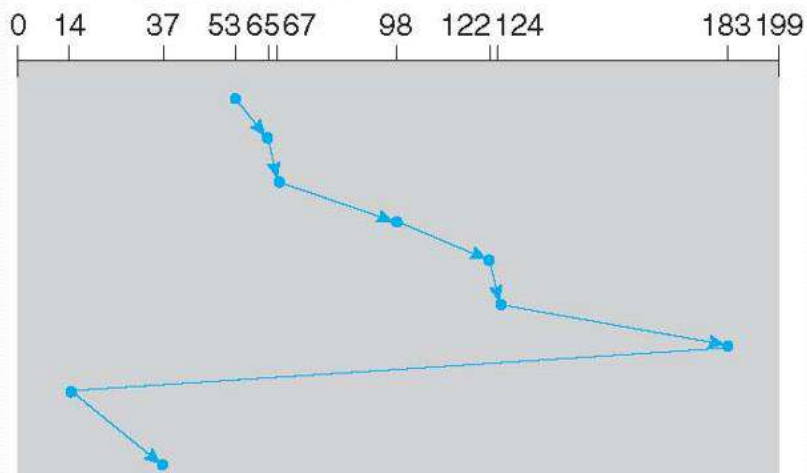
C-LOOK

- LOOK a version of SCAN, C-LOOK a version of C-SCAN
- Arm only goes as far as the last request in each direction, then reverses direction immediately, without first going all the way to the end of the disk

C-LOOK (Cont.)

queue = 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53



Selecting a Disk-Scheduling Algorithm

- SSTF is common and has a natural appeal
- SCAN and C-SCAN perform better for systems that place a heavy load on the disk
 - Less starvation
- Performance depends on the number and types of requests
- Requests for disk service can be influenced by the file-allocation method
 - And metadata layout
- The disk-scheduling algorithm should be written as a separate module of the operating system, allowing it to be replaced with a different algorithm if necessary
- Either SSTF or LOOK is a reasonable choice for the default algorithm

Swap-Space Management

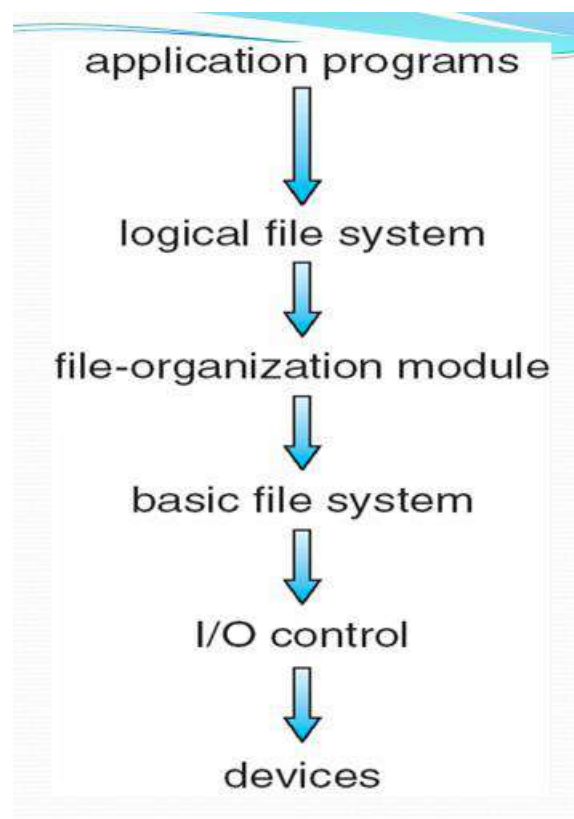
- Swap-space — Virtual memory uses disk space as an extension of main memory
 - Less common now due to memory capacity increases
- Swap-space can be carved out of the normal file system, or, more commonly, it can be in a separate disk partition (raw)
- Swap-space management
 - 4.3BSD allocates swap space when process starts; holds text segment (the program) and data segment
 - Kernel uses **swap maps** to track swap-space use
- Solaris 2 allocates swap space only when a dirty page is forced out of physical memory, not when the virtual memory page is first created
 - File data written to swap space until write to file system requested
 - Other dirty pages go to swap space due to no other home
 - Text segment pages thrown out and reread from the file system as needed

4) What is File System Structure and File Allocation Methods and File Access Methods and Free Space Management

Ans:-

File-System Structure

- File System provide efficient access to the disk by allowing data to be stored, located and retrieved in a convenient way. A file System must be able to store the file, locate the file and retrieve the file.
- Most of the Operating Systems use layering approach for every task including file systems. Every layer of the file system is responsible for some activities.



Layered File System

- **Device drivers** manage I/O devices at the I/O control layer. works as translator.
 - Given commands like “retrieve block no 123” outputs low-level hardware specific commands to hardware controller.
- **The basic file system** issues generic commands to the appropriate device drivers to perform the operations. Manages buffers and caches.
- **File organization module** understands files, logical address, and physical blocks
 - Translates logical block # to physical block #
 - Manages free space, disk allocation

File System Layers (Cont.)

- **Logical file system** manages metadata information
 - Metadata includes all of the file-system structure except the actual data.
 - Manages the directory structure to provide information to the file-organization module.
 - It maintains file structure via file-control blocks
 - Protection

File-System Implementation

- To implement the open and close system calls several on disk and in memory structures are used.
- **On disk structures are:**
- **Boot control block(per volume)** contains info needed by system to boot OS from that volume.
 - If the disk does not contain an operating system, this block can be empty.
 - In UFS, it is called the boot block. In NTFS, it is the partition boot sector.
- **Volume control** contains volume details
 - Total # of blocks, # of free blocks, block size, free block pointers, free FCB count FCB pointers.
 - UFS-superblock, NTFS-master file table.

File-System Implementation (Cont.)

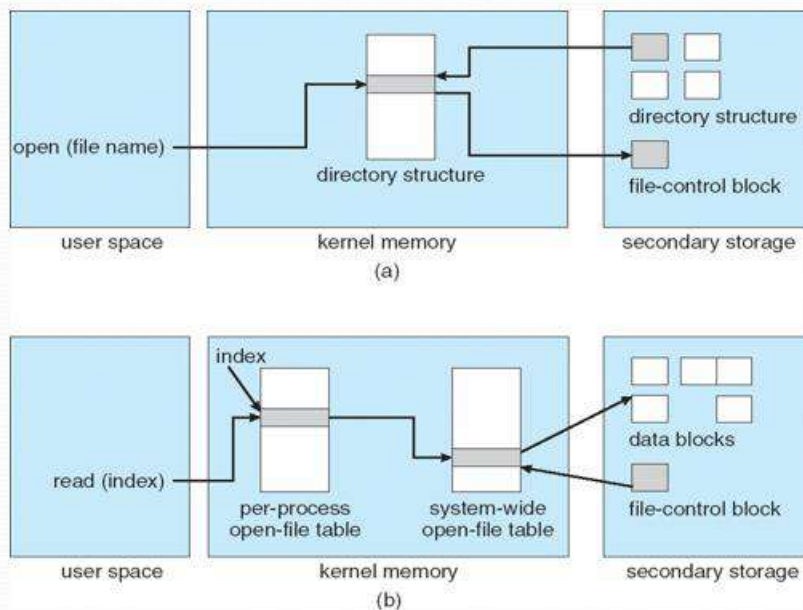
- **Directory structure**(per file system) :It is used to organizes the files,in UFS- File Names and inode numbers,NTFS- master file table
- Per-file **File Control Block (FCB)** contains many details about the file
 - inode number, permissions, size, dates
 - NFTS stores into in master file table using relational DB structures

file permissions
file dates (create, access, write)
file owner, group, ACL
file size
file data blocks or pointers to file data blocks

In-Memory File System Structures

- Data are loaded at mount time, updated during file-system operations and discarded at dismount.
- In memory **Mount table** stores info. about mounted volume.
- In memory **directory structure** cache holds the directory information of recently accessed directories.
- **System wide open file table:** contains a details of files which are opened by the processes.
- **Per process open file table:** each process stores information about files opened by the corresponding process.

In-Memory File System Structures



Directory Implementation

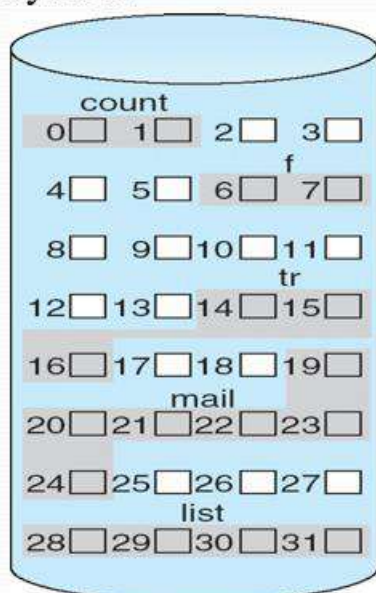
- **Linear list** of file names with pointer to the data blocks
 - Simple to program
 - Time-consuming to execute
- **Hash Table** – Linear list of file names with pointer to the data blocks but hashing is used for searching.
 - Decreases directory search time
 - **Collisions** – situations where two file names hash to the same location
 - Only good if entries are fixed size, or use chained-overflow method

Allocation Methods - Contiguous

- An allocation method refers to how disk blocks are allocated for files:
- 3 approaches: Contiguous, Linked, Indexed allocation.
- **Contiguous allocation** – each file occupies set of contiguous blocks
 - Best performance in most cases
 - Simple – only starting location (block #) and length (number of blocks) are required
 - Problems include finding space for file, knowing file size, external fragmentation, need for **compaction off-line (downtime)** or **on-line**
 - Sequential access and direct access can be supported.
 - Starting address+block number($b+i$)

Contiguous Allocation

- Mapping from logical to physical



directory

file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

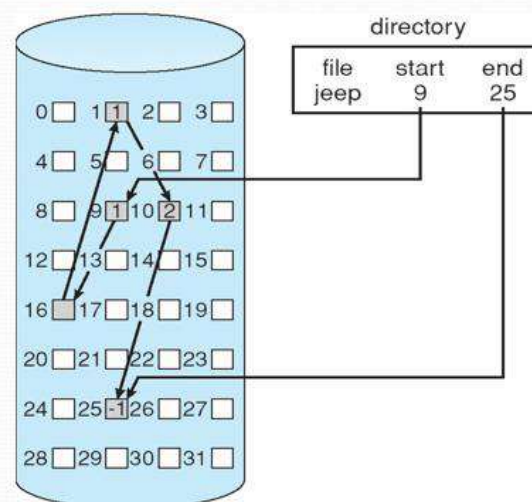
Allocation Methods – Linked

- **Linked allocation** – each file a linked list of blocks
 - File ends at null pointer
 - Each block contains pointer to next block
 - No compaction, no external fragmentation
 - Free space management system called when new block needed
 - Improve efficiency by clustering blocks into groups but increases internal fragmentation
 - Reliability can be a problem
 - Locating a block can take many I/Os and disk seeks

Linked Allocation

- Main disadvantage is good for only sequential access.
- Disadvantage is most of the block space used for linking, solution is to maintain the list of clusters instead of blocks.
- Each file is a linked list of disk blocks: blocks may be scattered anywhere on the disk.

Linked Allocation



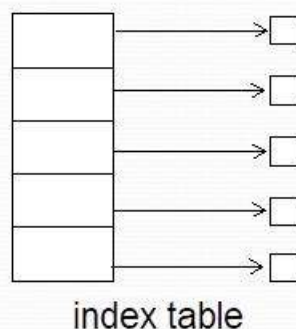
Allocation Methods – Linked (Cont.)

- FAT (File Allocation Table) variation
 - Beginning of volume has table, indexed by block number
 - Much like a linked list, but faster on disk and cacheable
 - New block allocation simple.
 - The directory entry contains the block number of the first block.
 - The table entry indexed by the block number contains the block number of the next block.
 - Last block contains special end of file value.
 - Adding a new file simply finding in the table index with value zero and add the index value to the end index and insert the special end of file value in the new index.

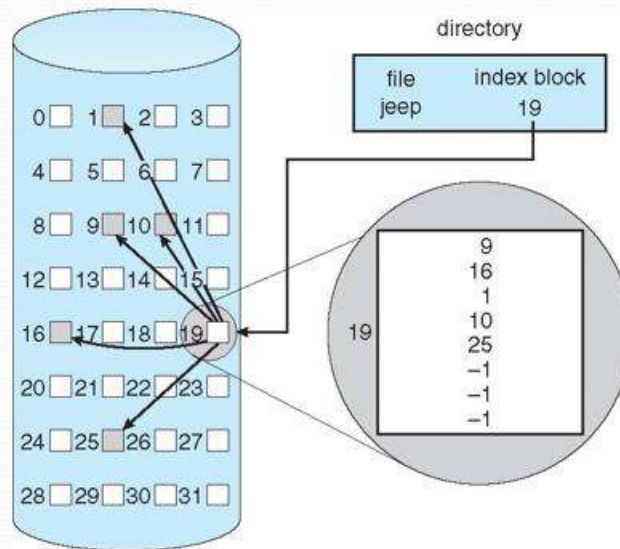
Allocation Methods – Indexed

- **Indexed allocation**
 - Each file has its own **index block(s)** of pointers to its data blocks

- Logical view



Example of Indexed Allocation



Indexed Allocation (Cont.)

- Need index table
- Random access
- Dynamic access without external fragmentation, but have overhead of index block

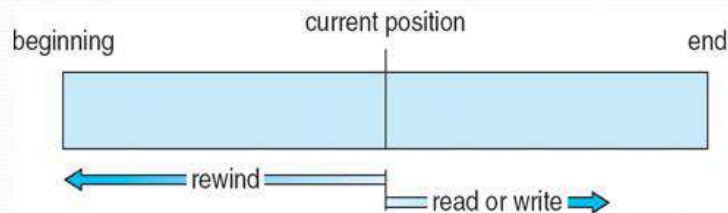
Access methods

- To access the information present in the file we have 3 methods
- **Sequential access**
- **Direct access/Relative access/Random Access**
- **Indexed sequential-access method**

Access methods

- Sequential access:

- Sequential access is the simplest method of the methods.
- Information in a file is accessed one after another.
- Accessing the file in sequential order, one record after another.



- Direct access:

- A file is made up of fixed length logical records.
- Allows to read and write at any record directly.
- For the operations the block number is specified as the parameter.
- The block number specified by the user is the relative block number i.e from starting of the file.
- Some operating systems allow both access methods, allows the user to specify the access type.

Access Methods

- Sequential Access read next(read pointer –next position)

write next
reset

Direct Access – file is fixed length **logical records**

read n
write n
position to n
read next
write next
rewrite n

n = relative block number

Simulation of Sequential Access on Direct-access File

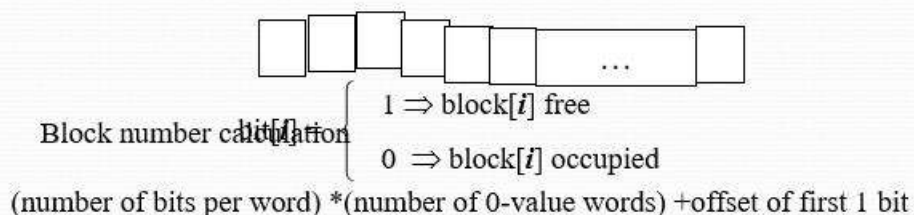
sequential access	implementation for direct access
<i>reset</i>	<i>cp = 0;</i>
<i>read next</i>	<i>read cp;</i> <i>cp = cp + 1;</i>
<i>write next</i>	<i>write cp;</i> <i>cp = cp + 1;</i>

Indexed sequential-access method

- An Index is created which contains key field and pointers to the various blocks.
- The records are stored in sequential order.
- It was developed by IBM.
- Searching for a specific record in a large database is relatively easy with ISAM.

Free-Space Management

- File system maintains **free-space list** to track free disk blocks/clusters.
- To create a file we search for free space. when a file is deleted free space is added to free space list.
- **Bit vector** or **bit map** (n blocks). Each block is represented by 1 bit.



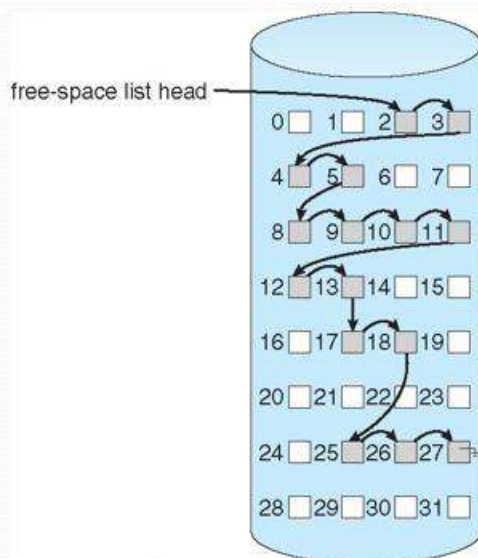
CPUs have instructions to return offset within word of first "1" bit

Free-Space Management (Cont.)

- Keeping the bit map in main memory is possible for smaller disks but may not be possible for larger ones.
- Bit map requires extra space
- Advantage-Easy to get contiguous files

Linked Free Space List on Disk

- Link together all the free disk blocks.
- Keeping a pointer to the first free block in special location on the disk.
- First block contains the pointer to the next free block and so on.
- Linked list (free list)
 - Cannot get contiguous space easily
 - No waste of space
 - No need to traverse the entire list (if # free blocks recorded).



Free-Space Management (Cont.)

- Grouping
 - Modify linked list to store address of next $n-1$ free blocks in first free block, plus a pointer to next block that contains free-block-pointers.
- Counting
 - Because space is frequently contiguously used and freed, with contiguous-allocation algorithm, extents, or clustering
 - Keep address of first free block and count of following free blocks.
 - Free space list then has entries containing addresses and counts.

5) What is protection? Goals and principle of protection and access matrix.

Ans:-

Goals of Protection

- Protection refers to a mechanism for **controlling the access of programs, processes, or users to the resources** defined by a computer system.
- Need to provide protection:
 - To prevent mischievous
 - Intentional violation of an access restriction by a user.
 - For a reliable system
- The role of protection in a computer system is to provide a **mechanism (how** will be done) for the enforcement of the **policies (what** will be done) governing resource use.
- Policies can be fixed in design or formulated by the management or designed by the users,
- Policies can be **dynamic**-so the application programmer must take care about the protection of the resources created and supported by it.

Principles of Protection

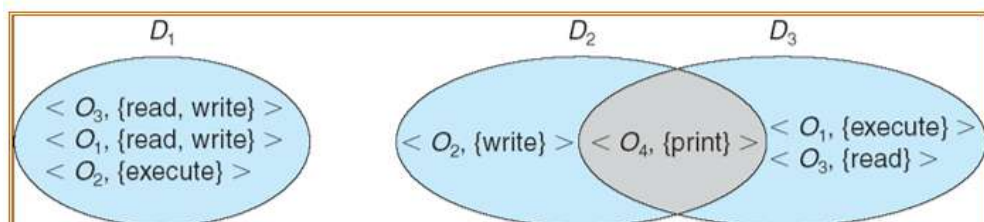
- A guiding principle for protection is :– **principle of least privilege**
 - which dictates that programs, users, and even systems be given just enough privileges to perform their tasks.
 - Failure results in minimum damage.
 - Managing users with the principle of least privilege entails (necessitates) **creating a separate account for each user**, with just the privileges that the user needs.-Role based access control(RBAC).
 - The principle of least privilege can help produce a more secure computing environment.

Domain of Protection

- A computer system is a collection of processes and objects.
- Objects are essentially abstract data types.
- The operations that are possible may depend on the object.

Domain Structure:

- A process operates with in a **protection domain**, which specifies the resources that the process may access.
 - Each domain defines a set of objects and the types of operations that may be invoked on each object.
 - The ability to execute an operation on an object is an **access right** .
 - Domains may be static or dynamic.
 - One way to provide dynamic access rights is to include all the rules initially in the domain which is not efficient.
 - Other way is to switch between the domains.
-
- A domain is a collection of access rights, each of which is an ordered pair
 $\langle \text{object-name}, \text{rights-set} \rangle$
-
- Ex: $\langle \text{file } F, \{\text{read}, \text{write}\} \rangle$
-
- Domain switching: enabling the process to switch from one domain to another.



A domain can be realized in a variety of ways:

- Each **user** may be a domain. In this case, the set of objects that can be accessed depends on the **identity of the user**.
- Each **process** may be a domain. In this case, the set of objects that can be accessed depends on the **identity of the process**.
- Each **procedure** may be a domain. In this case, the set of objects that can be accessed corresponds to the **local variables** defined within the procedure.

Access Matrix

- View protection as a matrix (*access matrix*)
- Rows represent domains
- Columns represent objects
- $Access(i, j)$ is the set of operations that a process executing in $Domain_i$ can invoke on $Object_j$
- The access matrix provides an appropriate mechanism for defining and implementing strict control for both the static and dynamic association between processes and domains.

Access Matrix

object domain	F_1	F_2	F_3	printer
D_1	read		read	
D_2				print
D_3		read	execute	
D_4	read write		read write	

Use of Access Matrix

- If a process in Domain D_i tries to do “op” on object O_j , then “op” must be in the access matrix.
- Processes should be able to switch from one domain to another.
- The ability to copy an access right from one domain of the access matrix to another is denoted by an “ * ” appended to the access right.

Access Matrix with *Copy Rights*

object domain	F_1	F_2	F_3
D_1	execute		write*
D_2	execute	read*	execute
D_3	execute		

(a)

object domain	F_1	F_2	F_3
D_1	execute		write*
D_2	execute	read*	execute
D_3	execute	read	

(b)

Access Matrix With Owner Rights

object domain	F_1	F_2	F_3
D_1	owner execute		write
D_2		read* owner	read* owner write
D_3	execute		
(a)			
object domain	F_1	F_2	F_3
D_1	owner execute		write
D_2		owner read* write*	read* owner write
D_3		write	write
(b)			

Modified Access Matrix

object domain	F_1	F_2	F_3	laser printer	D_1	D_2	D_3	D_4
D_1	read		read			switch		
D_2				print			switch	switch control
D_3		read	execute					
D_4	write		write		switch			

Implementation of Access Matrix

Access matrix will be generally a sparse matrix

object domain	F_1	F_2	F_3	printer
D_1	read		read	
D_2				print
D_3		read	execute	
D_4	read write		read write	

i) Global table

ii) Access Lists for objects

iii) Capability lists for Domains

iv) A Lock Key Mechanism

6) What is security? Types of security threats and implementing security defense.

Ans:-

The Security Problem

- ▶ We can say a system is **secure** if its resources are used and accessed as intended under all circumstances.
- ▶ **Intruders (crackers)** will try to attempt to breach security.
- ▶ **security breach** is any incident that results in unauthorized access to computer data, applications, networks or devices.
- ▶ **Threat** is potential security violation that means it will violate the security measures.
- ▶ **Attack** is the attempt to breach security.
- ▶ Accidental threat: Corruption of data caused by the programming error
- ▶ Malicious threat: Tampering with the system Eg: Trojan Horse, Viruses

The Security Problem

- ▶ **Attack** is attempt to breach security i.e., means gaining the access of data by unauthorized users.
- ▶ So unauthorized users can directly access the data, modify the data, destroy the data.
- ▶ There are two types of attacks:
 1. Passive attack
 2. Active attack

Passive Attack: The data will not be modified but the data will be accessed by the unauthorized users.

Active Attack: The data will be modified by the unauthorized users.

Threats

- ▶ Threats can be classified into two types:
 1. Program Threats
 2. System Threats

Program Threats:

A program written by a cracker to hijack the security or to change the behavior of anormal process

When any program is created by a user and if it is used by another user then misuse of the program may occur.

The Common way in which the behavior can be achieved are:

1. Trojan Horses.
2. Trap Door
3. Logic Bomb
4. Stack/buffer Overflow.
5. viruses

Program Threats

▶ Trojan Horse

- It is also like a virus.
- It is a piece of code/ hidden code which is attached to the software and reveals the confidential information.
- The functionality of the Trojan Horse is also different from Viruses and the Worms.
- It doesn't modify the program(Like Viruses) and it doesn't replicate itself(Like Worms).
- It is a hidden piece of code, which is intended to leak the confidential information.

▶ Trap Door:

- It is also called as a back door. Essentially it provides a secret entry point to a program or a system and this secret entry point is typically known to a attacker or a programmer.
- This backdoor is a program typically works by recognizing some special input commands such as sequence of input specify over a special user id. For example an attacker can gain access to a system through the back door without providing the proper user authentication.

Program Threats

▶ Logic Bomb

- It is basically a programming code that is intended into the program intentionally by the designer basically and the execution of that particular code will occur based on the functionality of the code written.
- Corruption of data caused by the programming error.

▶ Stack and Buffer overflow

- There is one program created by the user X, and that program gets installed on the Y machine, So the program created by X, that consumes all of the resources of Y.
- In this kind of situation, the buffer overflow can occur.

Program Threats

▶ Viruses

- It is a piece of code that attaches itself to a legitimate program code or an application and runs whenever that application runs.
- It intends to modify the program functionality.
- It executes without the user permissions.

categories of viruses

- File
- Boot
- Macro
- Source code
- Polymorphic
- Encrypted
- Stealth
- Tunneling
- Multipartite
- Armored

System and Network Threats

- ▶ In this operating system files or resources are misused.
- ▶ The common in which the behavior can be achieved are:
 1. Worms
 2. Denial of service
 3. Port scanning
- ▶ **Worms:**
 - It is also like a virus but the implementation is different.
 - Viruses modifies the program, but Worms doesn't modify the program.
 - They are the small computer programs that replicate (duplicate) itself again and again.
 - It contains malicious code that cause major damage to the operating system files.
 - It consumes lots of computer resources which makes the system to work slow.

System and Network Threats (Cont.)

- ▶ **Denial of Service**
 - Overload the targeted computer preventing it from doing any useful work.
 - The attacker is sending continuous request to the server. For example let us assume server limit is to access only 100 requests at a time.
 - As server is getting overloaded, so the authorized user is not going to get the service.
 - As server is continuously servicing the fake user, so for the authorized user it is not going to get the services.
 - Distributed denial-of service.

Implementing Security Defenses

- ▶ **Defense in depth** is most common security theory – multiple layers of security
- ▶ Vulnerability Assessment
- ▶ Security policy describes what is being secured
- ▶ Intrusion detection attempts to detect attacks both successful and unsuccessful attempts.
 - **Signature-based** detection spots known bad patterns
 - **Anomaly detection** spots differences from normal behavior can detect unknown attacks.
 - Can detect **zero-day** attacks
- ▶ Virus protection

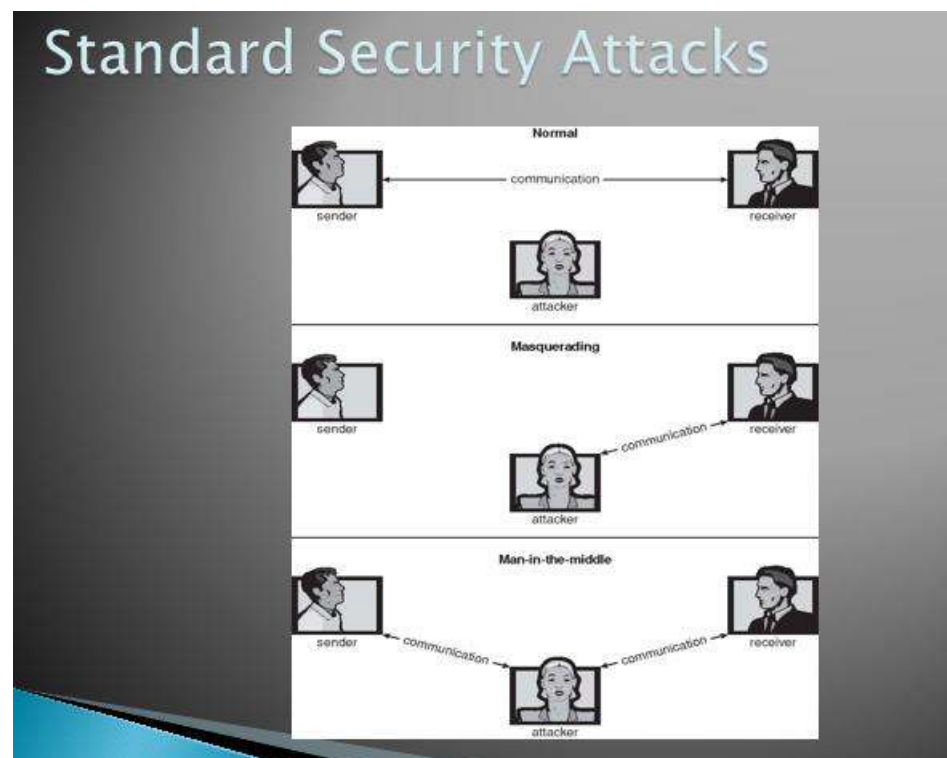
7) Different types of intruders.

Ans:-

Three Classes of Intruders

- **Masquerader** – unauthorized user who penetrates a system exploiting a legitimate user's account (*outside*)
- **Misfeasor** - legitimate user who makes unauthorized accesses or misuses his privileges (*inside*)
- **Clandestine user** - seizes supervisory control to evade auditing and access controls or suppress audit collection (*inside|outside*)

- ▶ **Intruders (crackers)** will try to attempt to breach security.
- ▶ **Masquerading** (breach **authentication**)
 - Pretending to be an authorized user to escalate privileges
- ▶ **Replay attack**
 - As is or with message modification
- ▶ **Man-in-the-middle attack**
 - Intruder sits in data flow, masquerading as sender to receiver and vice versa
- ▶ **Session hijacking**
 - Intercept an already-established session to bypass authentication



All the best

Now do the

Rest...